Agentic-AI Solution for an Institutional Process

Internship Report Submission and Verification

Submitted by: Pransh

Process selected: Internship Report Submission and Verification

Institution (fictionalised): Champa University

Course: Gen AI

1. Process Identification

1.1 The process

The process is **Internship Report Submission and Verification**, the reporting workflow a student follows during the summer internship. It runs from the first weekly progress report to the faculty-signed final report. The internship at Champa University lasts eight to twelve weeks depending on the company, and reporting has two parts: a recurring weekly submission and a single comprehensive report at the end. I am scoping the agent to that spine. Two adjacent stages, the mid-term presentation before a faculty panel and the final physical hard-copy signature, sit outside the agent's boundary and are noted but not automated.

1.2 Stakeholders

- **Student intern** produces the weekly and final reports, and is the primary user of the agent.
- **Industry mentor**, the company-side supervisor, validates that the reported work happened and signs each weekly report.
- **Faculty mentor**, on the institute side, is the reviewer, evaluator, and signing authority, and typically carries ten to fifteen interns at once.
- **Panel faculty**, two members distinct from the faculty mentor, assess the mid-term presentation. This sits adjacent to our scope.
- **Program Office** owns the template, tracks compliance, and maintains records.

1.3 Current workflow

The **weekly cycle** repeats through the internship. The student fills in the institute template, which carries two prompts, what was done and what was learned, plus signature blocks. The student gets the industry mentor's signature, signs personally, and emails the report to the faculty mentor. In principle the mentor reviews each one. In practice, a mentor carrying ten to fifteen interns cannot read every weekly log, so oversight shifts to a fortnightly or monthly video check-in where progress and problems are actually discussed. The weekly report quietly becomes a compliance formality rather than a live feedback loop.

The **final-report cycle** runs at the end. The student compiles a report that commonly exceeds fifty pages on average. Here a second problem surfaces. The institute wants assurance the work is the student's own, but the only authenticity tool, Turnitin, sits with faculty alone and enters only at the very end. The student never sees it and cannot self-check while writing. The signal itself is unreliable: different AI-detection tools return different verdicts on the same text, disagree about which passages look machine-written, and the free online options are close to useless. The authenticity judgment becomes a gamble on one inconsistent reading. Verification falls back on manual suspicion and repeated revision, cycling between student and mentor until the mentor digitally signs the soft copy. A physical hard copy, signed by the mentor, closes the process.

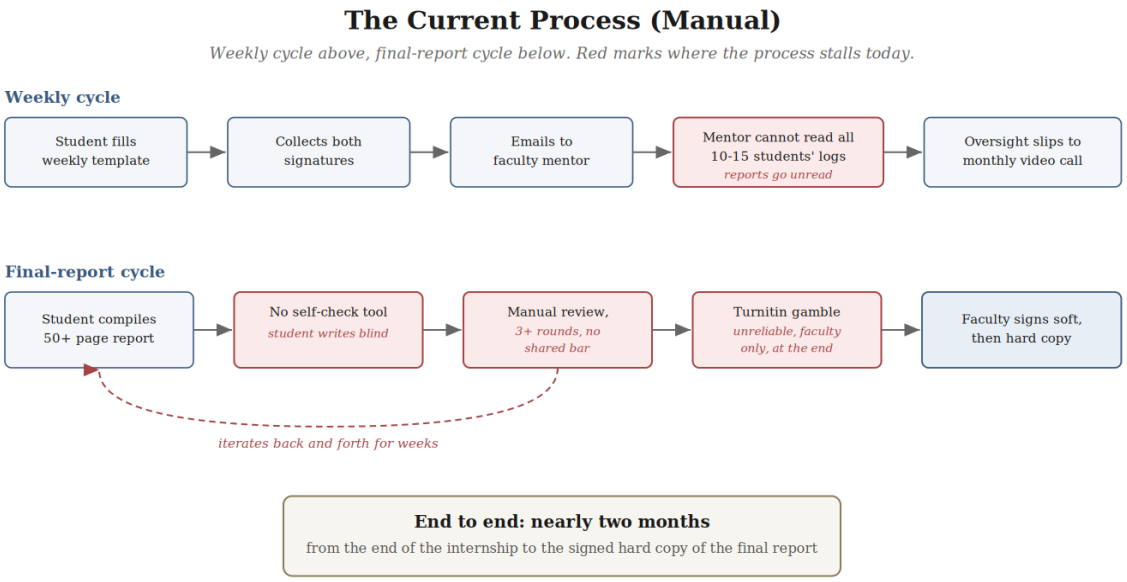


Figure 1. The current manual process, with the points where it stalls marked in red.

1.4 Pain points and bottlenecks

Signature collection is a manual, multi-party ritual repeated weekly. Email submission leaves no central record, no audit trail, and no visibility into who is falling behind until it is late. Faculty review does not scale: a mentor cannot read every weekly report for ten to fifteen students, so the artifact loses its purpose and oversight compresses into occasional calls.

The final report is harder. Review is slow, subjective, and inconsistent across mentors. The student has no way to test a draft against the institute's requirements before submitting, which forces

avoidable revision rounds. And the authenticity question stays unresolved throughout, because the one tool that exists is held only by faculty, applied only at the end, and cannot be taken at face value. The student, who most needs an early signal, has none. The net effect is high effort and long cycle time producing a compliance record rather than real feedback.

1.5 Why this needs an agentic solution rather than a form

A web form would fix the smallest part of this. It could capture submissions, store files, and route them by email, which is worth doing but leaves the actual work untouched. A form cannot read a fifty-page report and assess it against a rubric, reconcile the weekly reports against the final document for consistency, give the student a formative quality signal before submission, or draft rubric-aligned feedback for the mentor to edit.

One capability deserves emphasis, because it hits the central bottleneck. The agent can maintain an auto-updating dashboard for each faculty mentor, refreshed as reports arrive, showing all ten to fifteen interns with the metrics that matter: submission status and streaks, a short summary of each week's work, a consistency signal across the running narrative, and a flag on anything needing a human eye. This turns a stack of unread emails into one scannable view and restores the weekly report to something a mentor can act on.

These capabilities require reasoning over unstructured text, synthesis across documents, tool use such as parsing and rubric and similarity checks, planning across steps, and a clear sense of when to defer to the mentor. That is what defines an agentic solution and what a form lacks. The intent is not to replace the mentor's evaluation, nor to issue an automated verdict on authenticity, which no tool can honestly deliver given how much detectors disagree. Its role is narrower and more useful: remove the manual load, give the student an early self-check they never get today, and surface an evidence-backed view so the human decisions, the feedback and the signature, are faster and better informed.

2. Business Problem Statement

2.1 Problem statement

Champa University's internship reporting runs almost entirely by hand. Students submit by email, chase signatures week after week, and wait on manual review, all across an eight to twelve week internship. The weekly report was meant to be a feedback tool. In practice it has become a formality, because a faculty mentor with ten to fifteen interns cannot read every one. The final report is reviewed slowly and subjectively, with no shared quality bar and no way for the student to check their own work before handing it in. Authenticity is judged only at the end, using a single unreliable tool that only faculty can access. The process ends up costing real effort from students, industry mentors, and faculty, yet what it produces is a compliance record rather than genuine oversight or timely feedback.

2.2 Inefficiencies in the current process

The costs show up in both time and quality. On the numbers, figures marked as stated assumptions are illustrative and would be validated against Program Office data, while the two-month timeline is drawn from direct experience:

- A faculty mentor receives roughly **8 to 12 weekly reports per student**. Across a full mentee group that is **120 to 180 reports in one cycle** (stated assumption). Reading them all is not realistic, so most go unread.
- The **final report exceeds fifty pages on average** and usually goes through **three or more revision rounds** before sign-off (revision count is a stated assumption).
- From the end of the internship to the signed hard copy of the final report, the whole closing stage takes **nearly two months**. That is the single clearest sign of how much drag the manual process carries.
- Signature collection involves **three parties on every weekly report** and repeats each week, with no consolidated trail.

Beyond the numbers, four problems stand out. First, oversight is reactive: instead of catching issues as they happen, problems only surface at the occasional monthly call, often well after the fact. Second, review quality is inconsistent, because each mentor applies their own personal,

undocumented standard. Third, the student writes without any feedback signal, and only learns what fell short after submitting. Fourth, there is no system of record at all, so tracking and auditability depend on scattered email threads.

2.3 Value proposition

The agent goes after each of these directly. It takes on the manual load by reading submissions, checking they are complete, and drafting summaries and feedback for a human to review. It gives the weekly report its purpose back through a live faculty dashboard that makes an entire mentee group readable at a glance. It gives the student an early, honest check on quality and consistency before they submit, which cuts down revision rounds. And it creates a single record of everything, with a full audit trail, in place of the email-and-signature routine.

The payoff falls into four areas. On **time**, the agent shortens the review and revision cycle and clears the reading backlog that currently just goes undone, which frees faculty to spend their attention on the judgment only they can offer. On **quality and consistency**, it holds every student to the same rubric-based standard and finally gives the student the feedback loop the process was always meant to have. On **error reduction**, the routine checks that people currently do by hand and often skip, missing signatures, missed deadlines, incomplete sections, are done the same way every time, so nothing slips through unnoticed. And on **availability**, the agent works whenever the student does, so a draft submitted late at night gets its completeness check and its early feedback signal without waiting for a mentor to find the time. Through all four, every evaluative and signing decision stays with the faculty mentor. The agent is not there to replace human judgment. It clears away the manual work that currently crowds that judgment out.

3. Agent Design and Decision Boundaries

3.1 Design principle

The design starts from one observation about the process. The valuable work here is human judgment, and the expensive work is manual load, and right now the load is crowding out the judgment. Faculty mentors can evaluate an internship report well. What they cannot do is find the time to read every weekly log for ten to fifteen students, or absorb a fifty-page final report quickly enough to turn it around without weeks of delay.

So the agent is built to carry the load, not the judgment. It reads, checks, summarizes, and drafts. The faculty mentor still evaluates, signs, and decides. Every step that is computable is handled by ordinary code, and the language model is used only where real reading and reasoning are needed. This keeps the system cheap to run, easy to trust, and simple to defend, because the agent never makes an evaluative or irreversible call on its own.

3.2 System overview

The solution is a small multi-agent team coordinated by an orchestrator, arranged in a hub-and-spoke pattern. The orchestrator sits at the center and routes work based on what has just happened: a weekly report arrived, the final report arrived, a deadline is approaching, or a faculty mentor opened the dashboard. Each of those triggers runs a short, ordered workflow through the relevant specialists. The pattern fits the process, which has a few distinct entry points but a linear set of steps inside each one.

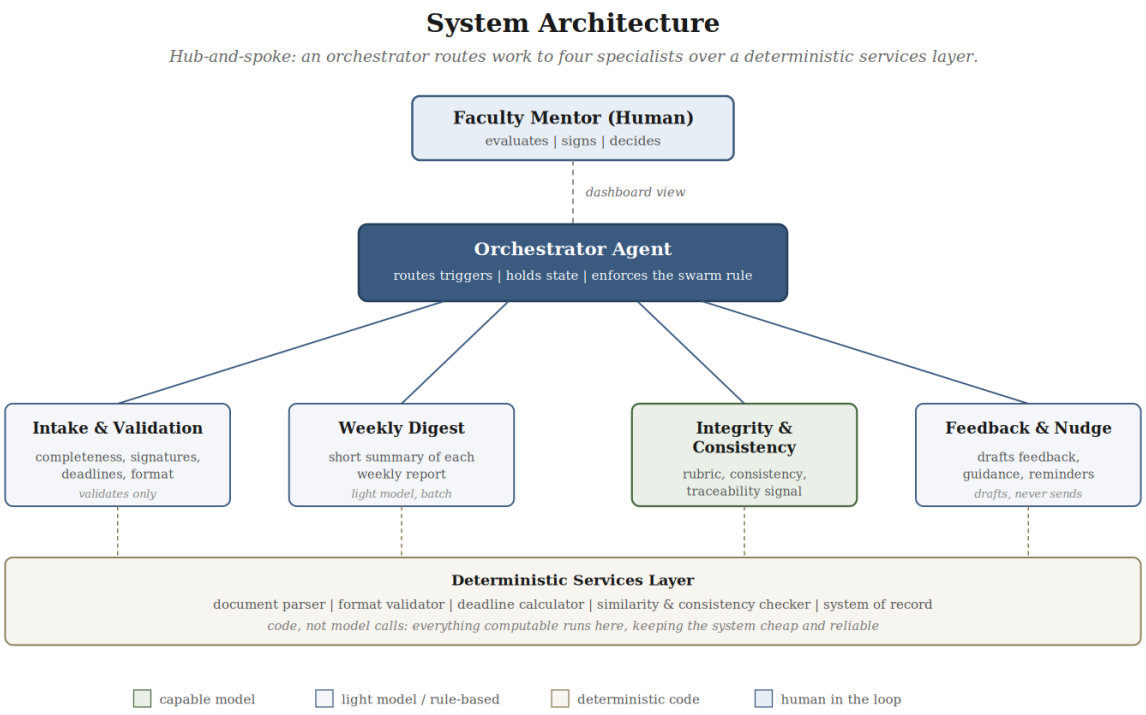


Figure 2. The agent team: an orchestrator routing work to four specialists over a deterministic services layer.

Four specialist agents sit around the orchestrator, supported by a deterministic services layer and a faculty dashboard.

Orchestrator Agent. Receives every trigger, routes it to the right workflow, holds the shared state, and enforces the one rule no single specialist can see on its own: a report is only marked ready for faculty signature once validation, digest, and integrity checks have all completed.

Intake and Validation Agent. The gatekeeper. It confirms that both template questions are answered, that the required signatures are present, that the submission is on time, and, for the final report, that the page count and template structure are met. Most of this is a straightforward field-and-format check. The language model is used only for genuinely unstructured cases, such as reading a scanned signature page. This agent validates. It never evaluates and never writes feedback.

Weekly Digest Agent. Reads each weekly report and produces a short structured summary of the work done, the learnings, and the main themes. It runs on a lighter, cheaper model and works in

scheduled batches, since weekly reports are short and nothing here is time-critical. Its summaries feed both the dashboard and the record used to check the final report later.

Integrity and Consistency Agent. The analytical core, and the only agent that needs a more capable model. It does three things. It checks that the weekly reports form a consistent narrative over time. It checks that the final report aligns with that weekly trail. And it checks the report against the institute's quality rubric. From these it produces a traceability signal on authenticity, described below. Everything it produces is a score or a flag for a human to read. It never takes an action.

Feedback and Nudge Agent. Drafts three kinds of message: feedback for the faculty mentor to review, formative guidance for the student, and deadline reminders. It drafts evaluative content but does not send it. Plain logistical reminders, such as a note that a weekly report is due tomorrow, can go out on their own under a set policy. Anything carrying evaluative content waits for faculty approval.

Dashboard. This is a deterministic view rather than a language model, and it is worth being clear about that. It gathers the outputs of the other agents into a single live screen for the faculty mentor, covering all ten to fifteen interns at once: who has submitted and who is falling behind, a short digest of each week's work, consistency flags, and anything that needs a human eye. It turns a backlog of unread email into one scannable page.

3.3 Inputs and outputs

Inputs. The completed weekly report template with its two answers and signatures, the final internship report, the institute's report template and quality rubric, the internship policy, submission deadlines, and the student's own history of prior weekly reports.

Outputs. A weekly digest per report, a validation result with any missing items, a consistency and quality read on the final report, an advisory traceability signal, draft feedback and guidance for human review, logistical reminders, a live faculty dashboard, and a complete record of the whole cycle with an audit trail.

3.4 How authenticity is handled

This deserves its own note, because it is the part most likely to be questioned. The agent does not try to detect AI writing. Detection tools are unreliable, they disagree with one another, and treating any single score as a verdict would be dishonest.

Instead, the agent uses the weekly reports as an evidence trail. Work that appears gradually across eight to twelve weekly logs, in the student's own developing account, is credible on its face. Work that surfaces fully formed in the final report with no trace in any weekly log is simply unevidenced, and that is a reason for the faculty mentor to have a conversation, not grounds for an accusation. This turns authenticity from a detection problem, which no tool solves well, into a traceability question, which the structure of the process answers naturally. The agent surfaces the evidence. The faculty mentor decides what it means.

3.5 Decision boundaries

The line between what the agent may do on its own and what stays with the human is the center of the design. The agent is allowed to act only where the action is safe, reversible, and non-evaluative.

The agent may do on its own	Reserved for the faculty mentor
Read and parse submissions	Evaluate or grade the report
Validate completeness, signatures, format, and deadlines	Digitally sign the weekly and final reports
Generate weekly digests	Accept and finalize the final report
Compute consistency, quality, and traceability signals	Decide any academic-integrity concern
Populate and refresh the dashboard	Approve evaluative feedback before it reaches the student
Draft feedback, guidance, and reminders	Waive or override a policy requirement
Send plain logistical reminders on its own	

The pattern is consistent throughout. The agent handles reading, checking, summarizing, and drafting. The faculty mentor keeps every judgment that carries weight: the evaluation, the

signature, and any decision about integrity. The agent's role on authenticity is deliberately the lowest-authority version possible. It shows the evidence and leaves the judgment where it belongs.

3.6 Build approach

The system is designed with a general-purpose AI assistant, used for the strategy, the agent logic, the prompts, and the drafting of the SKILL.md, and built in Antigravity, used to develop the agents, wire up the tools, and produce the dashboard. Keeping design and build in these two tools reflects how such a system would realistically be put together.

The efficiency of the design comes from a few deliberate choices. Everything computable runs as ordinary code rather than a model call. A lighter model handles the routine summaries and drafts, and the more capable model is reserved only for the integrity checks. Weekly reports are processed in batches rather than in real time, since speed is not the constraint here. Static material like the rubric and template is loaded once and reused. And faculty attention is spent only on what the agent flags, rather than on everything.

4. Information Requirements

4.1 Overview

For the agent team to work, each piece of information it draws on has to be identified along with where it comes from and why it is needed. The requirements fall into two groups. **Static data** is the reference material that rarely changes across a cycle, such as the report template, the quality rubric, and the internship policy. The agent reads this to know what correct and complete look like. **Dynamic data and tools** are the live inputs and the working capabilities the agent uses on each submission, such as the report being reviewed, the student's prior weekly reports, and the parsers and checkers that turn raw files into something the agent can reason over.

The distinction matters for the design. Static material is loaded once and reused across every student, which keeps the system cheap. Dynamic material is specific to each submission and is where the actual reasoning happens.

4.2 Static data

Information	Source	Purpose
Weekly report template	Program Office	Defines the required fields, the two prompts, and the signature blocks, so the Validation Agent knows what a complete weekly report must contain.
Final report template	Program Office	Sets the required structure, sections, and length of the final report, used to check format and completeness.
Quality rubric	Program Office (synthetic for this POC)	The shared checklist of what a strong report contains, used by the Integrity Agent to give consistent quality feedback in place of each mentor's private standard.
Internship policy	Program Office	States the rules the agent must respect, such as deadlines, minimum requirements, and what needs faculty approval, so the agent stays within institute norms.
Submission SOP	Program Office	The step-by-step procedure for how reports move through submission, review, and sign-off, used by the Orchestrator to route work correctly.
Feedback and email formats	Program Office / faculty convention	Standard formats for feedback notes and reminders, so the drafts the agent produces match how the institute already communicates.

4.3 Dynamic data and tools

Information or tool	Source	Purpose
Current report under review	Student submission	The live input the agent validates, digests, and checks.
Student's prior weekly reports	System of record	The evidence trail the Integrity Agent uses for consistency and traceability.
Signatures (industry and student)	Student submission	Verified as a completeness check. Digital by default, needing no interpretation, with a scanned page read only as a fallback.
Submission dates and deadlines	System of record	Used to compute on-time status, streaks, and reminders through date math, not a model call.
Document parser	Tool (built in Antigravity)	Converts PDF and DOCX files into clean text the agents can read.
Format validator	Tool	Checks page count, sections, and required fields against the templates, deterministically.
Similarity and consistency checker	Tool	Measures alignment between the final report and the weekly trail without an expensive model call. A text-overlap method in the first version, with sentence embeddings as the production path.
Rubric checker	Tool + Integrity Agent	Scores the report against the quality rubric to produce criterion-by-criterion feedback.
Notification drafter	Tool + Feedback Agent	Produces reminder and feedback drafts in the standard formats.
System-of-record database	Institute system	Stores every submission, digest, check, and decision, giving the process its audit trail.
Faculty dashboard	Built view	Presents aggregated status and flags for all ten to fifteen interns in one place.

4.4 A note on what the agent does not need

Two things are deliberately absent. The agent needs no external internet access for its core work, since everything it reasons over is internal to the institute, which keeps student data private and the system cheaper and safer. And it needs no AI-detection tool, because authenticity is handled through the internal traceability trail rather than an external detector. Keeping the information footprint this tight is a design choice in line with giving an agent only what it genuinely needs.

5. Agent Workflow

5.1 Overview

The system runs two main workflows, one for each kind of submission, plus two lighter triggers. Every workflow passes through the orchestrator, which decides what runs and in what order, and every workflow ends with something placed in front of the faculty mentor rather than a decision taken on the student's behalf. The steps below describe what happens from the moment a student submits to the moment the mentor acts.

5.2 Weekly report workflow

This runs each week, once per submission.

1. The student submits the weekly report. The orchestrator picks it up and starts the weekly workflow.
2. The Intake and Validation Agent parses the file and checks it: both questions answered, signatures present, submitted on time. If anything is missing, the workflow stops here. The student is told what is missing, and nothing further runs on an incomplete report.
3. Once the report is complete, the Weekly Digest Agent reads it and writes a short digest of the work done, the learnings, and the main themes. The digest is saved to the record.
4. The Integrity Agent does a light pass, checking that this week fits the running narrative from earlier weeks. It does not do the full quality review at this stage.
5. The dashboard updates. The mentor now sees this student's latest status, digest, and any consistency flag alongside the rest of the group.

The student's effort ends at step 1. Everything after is automatic, and the mentor sees the result without having to open an email.

5.3 Final report workflow

This runs once, at the end of the internship.

1. The student submits the final report. The orchestrator starts the final workflow.

2. The Intake and Validation Agent parses and checks it: template structure, page count, required sections, signatures. If anything is missing, the workflow stops and the student is told what to fix.
3. Once complete, the Integrity Agent runs its full pass. It scores the report against the quality rubric, checks it for internal consistency, and compares it against the trail of weekly reports to see how well its content is supported. From this it produces the quality scores and the traceability signal.
4. The Feedback Agent drafts two things from the integrity output: feedback for the mentor and formative guidance for the student. Both are drafts. Neither is sent.
5. The mentor opens the dashboard, reads the scores, the traceability signal, and the drafts, and decides. The mentor edits or approves the feedback, judges anything the traceability signal has flagged, and, when satisfied, signs the report. The signature is the mentor's, not the agent's.

The pattern is the same as the weekly workflow but with a fuller check and a human decision at the end. The agent does the reading and the drafting. The mentor does the judging and the signing.

5.4 The two lighter triggers

Two smaller workflows run outside the submission flow. When a **deadline is approaching**, the Feedback Agent drafts a reminder. If it is a plain reminder, dates and status only, it goes out on its own. Anything with evaluative content waits for the mentor. When the **mentor opens the dashboard**, the orchestrator serves the latest aggregated view across all ten to fifteen interns, no processing needed, just the current state of the record.

5.5 Why the workflow is shaped this way

Two things hold across every workflow. First, an incomplete submission never moves forward, which stops the agent from wasting work on a report that is not ready and stops a half-finished report from reaching the mentor. Second, the workflow always ends at a human for anything that carries weight. The agent narrows the mentor's work to a decision rather than a search, but it never makes that decision. This is the same boundary the whole design rests on, expressed as a sequence of steps.

6. Skill Folder Design

6.1 Approach

Since the system runs on more than one agent, a single flat Skill folder would not hold it cleanly. I have split it into a coordinating top level plus one self-contained Skill folder per specialist agent underneath.

The split is deliberate. Anything shared, the institute policy, the templates, the rubric, sits once at the top level, so it is stored in one place and every agent reads from the same copy. Anything specific to one agent stays inside that agent's own folder. This keeps the design cheap to run and easy to maintain, because a change to a shared policy happens in exactly one file, and any single agent can be reworked without touching the rest.

6.2 Folder structure

```
internship-report-agent/
  Agents.md  orchestrator instructions
  Skillregistry.md  directory of specialists
  Summary.md  consolidated cycle record
  README.md  how to run the folder
  data/  input reports as submitted
  references/  shared policy, templates, rubric, SOP
  assets/  output templates
  scripts/  shared deterministic tools
  skills/  one Skill folder per specialist
  README.md  index of the four specialists
  intake-validation/  SKILL.md + example
  weekly-digest/  SKILL.md + style guide + example
  integrity-consistency/  SKILL.md + scoring guide + script + example
  feedback-nudge/  SKILL.md + tone guide + example
  output/  stage outputs and the audit trail
  tests/  test suite and fixtures
```

6.3 What goes where, and why

references/ (shared and per-agent). This holds the material the agents read to know what correct and complete look like: the institute policy, both templates, the quality rubric, and the submission SOP. The shared ones sit at the top level because every workflow touches them. Anything used by

just one agent, like the rubric-scoring guide the Integrity Agent needs, stays inside that agent's own folder. The reason policy lives here and not inside the instructions is practical: a policy changes often, the procedure rarely does, so keeping them apart means a policy update touches one reference file and leaves the agent logic alone.

assets/. These are the output templates the agents fill in, the weekly-digest format, the faculty-feedback format, and the reminder format. Keeping them separate from the logic lets the institute change how a feedback note or reminder looks without touching how the agent decides what to write.

scripts/. The deterministic tools live here: the document parser, the format validator, the deadline-and-streak calculator, and the similarity checker. These are the jobs that are purely computable, and doing them in code rather than through a model call is what keeps the system both cheap and reliable. The agent instructions just call these when needed.

output/. Every stage writes its result here, and together those results become the audit trail the current process is missing. Validation results, weekly digests, integrity reports, draft feedback, and dashboard data each get their own sub-folder, and Summary.md pulls the whole cycle together into one record the faculty mentor can read at the end.

data/. The reports as submitted, weekly and final, sit here, kept apart from everything the agent produces so an input is never confused with an output.

Agents.md and Skillregistry.md. These run the coordination. Agents.md tells the orchestrator how to route each trigger, and Skillregistry.md is the lookup that tells it which specialist owns which task.

6.4 How the structure supports the design

The layout follows straight from the decisions already made. Shared material sits once at the top so it is loaded once and reused, which is the cost discipline from Section 3. Each specialist is self-contained, so it can be reworked or reasoned about on its own, which was the whole reason for using separate agents instead of one overloaded prompt. And the output folder finally gives the process the system of record and audit trail whose absence was one of the original pain points. The

structure is not just a place to keep files. It carries the same separation of concerns the agent design rests on.

7. SKILL.md Design

7.1 What this section contains

A multi-agent system does not have a single SKILL.md. The orchestrator has its own instruction file, and each of the four specialists has its own. This section presents the orchestrator's routing first, then the complete SKILL.md for every specialist. These are first working versions, meant to be refined once tested.

7.2 Orchestrator

The orchestrator is defined by two small files. Agents.md holds the routing, shown below, and Skillregistry.md is a lookup of which specialist owns which task. One rule sits above the routing and lives here because no single specialist can see it: a report is marked ready for faculty signature only after validation, digest, and integrity have all completed.

```
# Orchestrator: Internship Report Agent

## Triggers and routing
- Weekly report submitted  -> Intake & Validation
  - if incomplete -> stop, record missing, notify student
  - if complete  -> Weekly Digest -> Integrity (light)
- Final report submitted  -> Intake & Validation
  - if incomplete -> stop, record missing, notify student
  - if complete  -> Integrity (full) -> Feedback (draft)
- Deadline approaching  -> Feedback (reminder only)
- Faculty opens dashboard  -> Serve latest aggregated view

## Shared rule
A report is ready for faculty signature only after Intake &
Validation, Weekly Digest, and Integrity have all completed.

## Boundaries
The orchestrator routes and tracks. It does not evaluate,
sign, or send evaluative content.
```

7.3 Intake and Validation Agent

The gatekeeper. It checks that a submission is complete and on time, and stops an incomplete one before it goes further. It validates only, and never judges quality.

```
---
name: intake-validation
description: Validates that an internship report submission is
  complete. Checks the two template questions are answered,
  signatures are present, the deadline status, and, for a final
  report, page count and template structure. Use when a weekly
  or final report is first submitted. Validates only, does not
  evaluate.
allowed-tools: Bash(python:*)
---

# Intake and Validation Agent

## When invoked
Check that a submission is complete and on time.

## Instructions
1. Parse the submitted file with scripts/document_parser.py,
  then read the relevant template from references.
2. Confirm both questions are answered.
3. Confirm the required signatures are present. Digital
  signatures need no interpretation. Read a scanned signature
  page only if that is what was sent.
4. Check the submission against the deadline using
  scripts/deadline_calculator.py.
5. For a final report, check page count and structure with
  scripts/format_validator.py.
6. Write a validation result listing anything missing.

## Guardrails
- Report completeness only. Do not judge quality or content.
- If something is missing, record it. Do not reject the
  submission yourself.

## Example
See examples/sample-validation.md for the shape of a validation
result, showing both a complete report that passes and an
incomplete one that stops the workflow.
```

7.4 Weekly Digest Agent

Reads a validated weekly report and writes a short, faithful summary for the dashboard and the integrity trail. It summarizes only, and does not evaluate.

```
---
name: weekly-digest
description: Summarizes a weekly internship report into a short
  structured digest of work done, learnings, and key themes.
  Use when a weekly report has passed validation. Summarizes
  only.
allowed-tools: Bash(python:*)
---

# Weekly Digest Agent

## When invoked
Produce a short digest of one weekly report.

## Instructions
1. Read the validated weekly report from the data folder.
2. Read the digest style guide from
  references/digest-style-guide.md.
3. Summarize the work done and the learnings in a few lines each.
4. Note the main themes of the week.
5. Write the digest to output/weekly-digests using the template
  in assets.

## Guardrails
- Summarize faithfully. Do not evaluate the quality of the work.
- Keep the digest short. It feeds the dashboard and the
  integrity trail.

## Example
See examples/sample-digest.md for a worked example showing a
weekly report and the digest produced from it.
```

7.5 Integrity and Consistency Agent

The analytical core, and the only agent on the more capable model. It scores a report against the rubric, checks the weekly narrative holds together, and produces the traceability signal. It carries the fullest frontmatter, with license and metadata, and it outputs scores and flags for a human, never actions.

```
---
name: integrity-consistency
description: Checks an internship report for quality against
  the institute rubric, for narrative consistency across the
  weekly reports, and for alignment between the final report
  and the weekly trail. Produces quality scores and an advisory
  traceability signal for the faculty mentor. Use when a weekly
  or final report needs a consistency, quality, or traceability
  check. Does not grade, does not accuse, and does not take
  action.
license: MIT
metadata:
  author: Champa University Program Office
  version: "1.0"
allowed-tools: Bash(python:*)
---

# Integrity and Consistency Agent

## When invoked
Assess an internship report for quality and consistency, and
produce an evidence-based traceability signal, all for a human
to review.

## Instructions
1. Read the quality rubric from references/quality-rubric.md
  and the scoring guide from references/rubric-scoring-guide.md.
2. Read the report under review from the data folder.
3. Read the student's prior weekly reports from
  output/weekly-digests.
4. Score the report against each rubric criterion.
5. Check the weekly reports for a consistent narrative over time.
6. For a final report, compare it against the weekly trail using
  scripts/similarity_checker.py to see how well its content is
  supported.
7. Produce a traceability signal describing how well the report
  is evidenced by the weekly trail.
8. Write the result to output/integrity-reports as scores and
  flags only.

## How to read the traceability signal
- Well evidenced: the work appears across the weekly reports in
  the student's own developing account. Treat as credible.
- Thinly evidenced: parts of the final report have little or no
  trace in any weekly report. Flag for the faculty mentor.
- Do not label any report as AI-written. Detection is
  unreliable. Report only how well the work is evidenced, and
  leave the judgment to the faculty mentor.
```

```
## Guardrails
- Output scores and flags only. Never grade, accept, reject,
  or sign.
- Never send anything to the student directly.
- If the weekly trail is missing or unreadable, say so and do
  not infer a traceability signal from absence alone.
- Escalate a thinly evidenced report to the faculty mentor.

## Example
See examples/sample-integrity-report.md for the shape of a
completed integrity report.

## Output
A short integrity report: rubric scores by criterion, a
consistency note, and a traceability signal with the evidence
behind it. All for faculty review.
```

7.6 Feedback and Nudge Agent

Drafts the feedback, guidance, and reminders. The boundary is enforced here: evaluative content is only ever a draft for the mentor, while a plain reminder may go out on its own.

```
---
name: feedback-nudge
description: Drafts faculty-facing feedback, student-facing
  formative guidance, and deadline reminders. Use when a report
  has been checked or a deadline is near. Drafts evaluative
  content but does not send it.
allowed-tools: Bash(python:*)
---

# Feedback and Nudge Agent

## When invoked
Draft feedback, guidance, or a reminder, depending on the
trigger.

## Instructions
1. Read the integrity report or the deadline status.
2. Read the feedback tone guide from
  references/feedback-tone-guide.md.
3. Draft faculty feedback and student guidance using the
  templates in assets.
```

```
4. Draft a reminder if a deadline is near.
5. Write drafts to output/draft-feedback.

## Guardrails
- Evaluative content is a draft only. It waits for faculty
  approval before it reaches the student.
- Only plain reminders, dates and status with no evaluative
  content, may be sent automatically.

## Example
See examples/sample-drafts.md for worked examples of faculty
feedback, student guidance, and a plain reminder.
```

7.7 Design notes

Three choices run through all of these files. The instructions say what to do without over-specifying how, because the agent can work out the obvious details and over-specifying only makes the file brittle. Policies and standards are read from reference files rather than written into the instructions, so a policy change touches one file and leaves the agent logic alone. And every file carries its own guardrails, which sit at three levels together: each agent's own limits inside its file, the handoff rule that the Integrity Agent passes evidence rather than a bare verdict, and the system-wide rule in the orchestrator that a report is ready only when every check is done. The result keeps each agent narrow, honest about what it can and cannot claim, and safe by default.